Patterns for OpenEdge Application Development

XML Mapping

John Sadd, Fellow
Progress Software Corporation

PROGRESS
SOFTWARE

Contents

DISCLAIMER **3**

XML MAPPING: DEFINITION **4**

HOW IT WORKS **4**

WHEN TO USE IT **4**

EXAMPLES **5**

USING XML AS AN UNMANAGED DATA STORE 5
SAVING CONTEXT DATA AS XML 9

SUMMARY **11**

Disclaimer

Certain portions of this document contain information about Progress Software Corporation's plans for future product development and overall business strategies. Such information is proprietary and confidential to Progress Software Corporation and may be used by you solely in accordance with the terms and conditions specified in your PSDN Subscription End User License Agreement. Progress Software Corporation reserves the right, in its sole discretion, to modify or abandon without notice any of the plans described herein pertaining to future development and/or business development strategies.

XML Mapping: Definition

Using the XML standard representation format as either a source for data to be imported into a ProDataSet in an OpenEdge application, or as a target for ProDataSet data for storage outside the database or for communication with other applications.

OpenEdge Release 10.1A supports the automated conversion of data in a ProDataSet or temp-table to XML, and the import of XML into a ProDataSet or temp-table. This support facilitates the coding of a number of ways in which an application may need to share or persist data. This paper briefly explores two of these uses as updates to earlier papers in the series on *Implementing the OpenEdge Reference Architecture*: using data from an unmanaged data store, and persisting ProDataSet data as context information.

How It Works

OpenEdge Release 10.1 supports four new methods on either a temp-table handle or a ProDataSet handle:

- **READ-XML** takes data in XML format from a file, a LONGCHAR, X-DOCUMENT, or MEMPTR and loads it into the temp-table or ProDataSet. Optionally the method can also use associated XML Schema information either to fill out the temp-table or ProDataSet definition if the object has no definition, or to verify the compatibility of the XML data with the temp-table or ProDataSet definition.

- **READ-XMLSCHEMA** creates or verifies the XML Schema information without loading any data.

- **WRITE-XML** takes data in a temp-table or ProDataSet and converts it to an XML document, which can be written to a file, a LONGCHAR, stream, X-DOCUMENT, or MEMPTR. Optionally the method can also create XML Schema information from the temp-table or ProDataSet definition.

- **WRITE-XMLSCHEMA** creates the XML Schema information without converting any data.

You can learn more about these methods in the chapter on *Reading and Writing XML Using Temp-Tables and ProDataSets* in the *OpenEdge Development:Programming Interfaces* book. The online help for the four methods details their full syntax and usage.

When to Use It

XML has two fundamental values as a data format.

First, XML represents data strictly in character format. This means that you can use XML to hold data independent of the need for an OpenEdge database or any other specific data management system. This paper shows how to take advantage of this to store ProDataSet data as state or context information between sessions or between interactions involving multiple sessions, without having to resort to converting the data to RAW database fields or other proprietary data formats.

Second, XML is a standard interchange format for data. This means that if you need to share data between your application and other non-OpenEdge applications or tools, it is likely that you can (and probably must) use XML as the way to represent data you send to those other programs, and as the format in which you receive data from them. In the context of the OpenEdge Reference Architecture, this non-OpenEdge data format represents an unmanaged data store. This paper explores briefly how to create and read flat files that represent an intermediary between your OpenEdge application and such an unmanaged store.

Examples

Using XML as an Unmanaged Data Store

The white paper entitled *Using an Unmanaged Data Source* in the series on *Implementing the OpenEdge Reference Architecture* describes how to create a specialized Data-Source object that gets its data from an XML source. The examples in that paper show how to use the OpenEdge support for the SAX parser to turn an XML document into OpenEdge data you load into a ProDataSet. The first example in this paper shows how the new support for the WRITE-XML and READ-XML methods in OpenEdge 10.1 greatly simplifies the coding needed to interface to XML data.

First you can write the contents of a ProDataSet to a file as XML. The example procedure writeOrder.p is simplified to the extent that the code to define the Data-Sources and load the data from the database into a ProDataSet is shown inline. You can of course also use this technique with Data Access Objects and their Data-Source objects as described in other papers in the Reference Architecture series.

The example uses these two temp-tables:

– The eOrder temp-table definition, with Order fields, along with the Customer Name and SalesRep Name and a calculated Order Total, defined in the include file etOrder.i;

– The eOrderLine temp-table definition for OrderLine fields, defined in the include file etOrderLine.i.

The dsOrder ProDataSet definition combines these two tables (leaving out the eItem table used in other examples to reduce the amount of data).

```
/* writeOrder.p */

  {etOrder.i}
  {etOrderLine.i}

  DEFINE DATASET dsOrder FOR eOrder,eOrderLine
        DATA-RELATION OrderLine FOR eOrder,eOrderLine
            RELATION-FIELDS (OrderNum,OrderNum).
```

Further definitions provide the procedure with buffer handles for the two tables, a query for retrieving the top-level Order, and Data-Sources for the two tables.

```
    DEFINE VARIABLE hOrderBuf AS HANDLE     NO-UNDO.
    DEFINE VARIABLE hOlineBuf AS HANDLE     NO-UNDO.
    DEFINE QUERY OrderQuery FOR Order,Customer,SalesRep.
    DEFINE DATA-SOURCE srcOrder FOR QUERY OrderQuery.
    DEFINE DATA-SOURCE srcOline FOR OrderLine.
```

New shorthand syntax available in Release 10.1 lets you easily obtain the buffer handles for the ProDataSet's two temp-tables:

```
    hOrderBuf = DATASET dsOrder::eOrder.
    hOlineBuf = DATASET dsOrder::eOrderLine.
```

Next the procedure attaches the two data sources, doing necessary field mapping for eOrder, and provides an event handler (also in writeOrder.p) to calculate the OrderTotal field for the Order after the OrderLines are read.

```
    hOrderBuf:ATTACH-DATA-SOURCE(DATA-SOURCE srcOrder:HANDLE,
                                 "Customer.name,CustName").
    hOlineBuf:ATTACH-DATA-SOURCE(DATA-SOURCE srcOline:HANDLE,"").
    hOlineBuf:SET-CALLBACK-PROCEDURE
        ("AFTER-FILL","eOrderLineAfterFill").
```

Finally the setup code prepares a query to retrieve data for Order number 1, and uses the FILL method to retrieve all the related data and load it into the ProDataSet:

```
    QUERY OrderQuery:QUERY-PREPARE
        ("FOR EACH Order WHERE Order.orderNum = 1,
        FIRST customer OF order, FIRST SalesRep OF order").
    DATASET dsOrder:FILL().
```

Now you can easily turn that data into an XML document using the WRITE-XML method.

```
    DATASET dsorder:WRITE-XML
    ("FILE",
     "dsOrder.xml",
     TRUE).   /* formatted */
```

The first argument is the target type; in this case you are writing the XML to an operating system file.

The second argument is the file name. This can have a relative or absolute pathname if needed.

The third argument tells Progress to insert white space and other formatting into the document for readability.

There are other optional arguments that the example doesn't use that are described in the product documentation. You can look at this excerpt from the output from writeOrder.p to see the format of the XML that WRITE-XML generates:

```
<?xml version="1.0" ?>
- <dsOrder xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
- <eOrder>
  <Ordernum>1</Ordernum>
  <CustNum>53</CustNum>
  …
  <CustName>Offside Hockey</CustName>
  <RepName>Robert Roller</RepName>
  <OrderTotal>4402.02</OrderTotal>
  </eOrder>
- <eOrderLine>
  <Ordernum>1</Ordernum>
  <Linenum>1</Linenum>
  <Itemnum>54</Itemnum>
  <Price>4.86</Price>
  …
  </eOrderLine>
```

The converse of this write operation turns the XML document into an unmanaged data source for the dsOrder ProDataSet. The procedure readOrder.p uses the READ-XML method to load the XML into the ProDataSet.

```
/* readOrder.p */

  {etOrder.i}
  {etOrderLine.i}

  DEFINE DATASET dsOrder FOR eOrder,eOrderLine
        DATA-RELATION OrderLine FOR eOrder,eOrderLine
            RELATION-FIELDS (OrderNum,OrderNum).

   DATASET dsorder:READ-XML
    ("FILE",
     "dsOrder.xml",
```

```
    "EMPTY",    /* read mode */
    ?,          /* schema location */
    FALSE).     /* override-default-mapping */
```

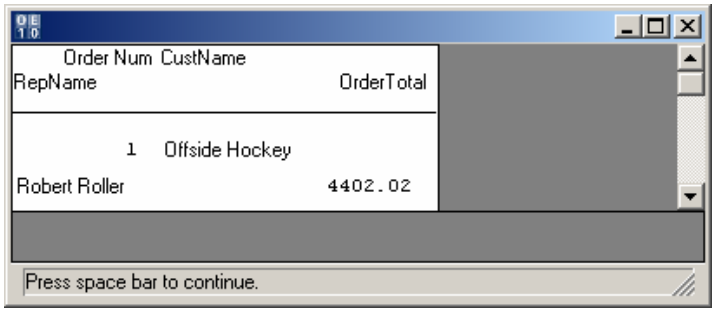
The first two arguments to READ-XML are the same as for WRITE-XML.

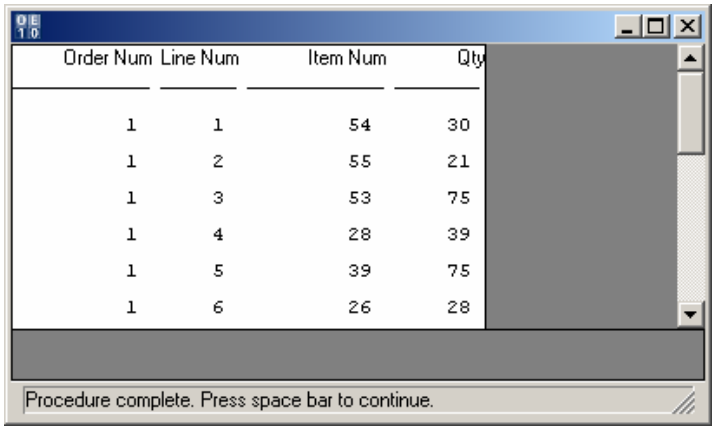
The third argument is the read mode, corresponding to the FILL-MODE attribute of a ProDataSet's temp-table buffers. The value "EMPTY" tells OpenEdge to empty the ProDataSet of any data before loading the new data.

The fourth argument is the location of an XML Schema document that describes the schema for the data. You can specify this if the read procedure does not have a static ProDataSet definition, which provides a known schema.

The final required argument tells OpenEdge whether to map string and binary values in the XML Schema (if supplied) to CHARACTER and RAW fields in the ProDataSet (if the argument is FALSE) or to CLOB and BLOB fields (if the argument is TRUE).

After the READ-XML statement executes, you can verify that the XML has been properly loaded back into the dsOrder ProDataSet:





If you look back at the XML for dsOrder, as generated by writeOrder.p, you will notice that the OrderLines are not nested within the Order. If there were multiple Orders, all the Order data would appear first in the XML, followed by all of the OrderLines. Typically this is not the way you want to generate an XML document; instead the child data should be nested within its parent, to represent the parent-child hierarchy in the XML.

To get this behavior, OpenEdge 10.1 supports a new NESTED keyword on the ProDataSet Data-Relation definition. This is also a logical Data-Relation attribute that you can set at runtime.

A variant of writeOrder.p called writeOrderNested.p shows this keyword at the end of the ProDataSet's Data-Relation definition for the OrderLine relation:

```
    DEFINE DATASET dsOrder FOR eOrder,eOrderLine
        DATA-RELATION OrderLine FOR eOrder,eOrderLine
            RELATION-FIELDS (OrderNum,OrderNum) NESTED.
```

When you write data out in this nested form, it is also likely that the application that receives the data will not expect the child Data-Relation fields to be written out as elements of the child object. In the case of the Order/OrderLine data, the child field is eOrderLine.OrderNum. In the relational environment of the OpenEdge application, this foreign key field is needed to relate OrderLines to their Orders. But in the hierarchical format of the XML, the value isn't needed, and you may want to suppress it from the output. You can do this by including the XML-NODE-TYPE attribute on the temp-table field definition, giving it a value of "HIDDEN", as shown in this excerpt from the include file variant etOrderLineNested.i, used by writeOrderNested.p:

```
DEFINE TEMP-TABLE eOrderLine BEFORE-TABLE eOrderLineBefore
   FIELD Ordernum AS INTEGER LABEL "Order Num" FORMAT "zzzzzzzzz9" INITIAL "0"
      XML-NODE-TYPE "HIDDEN"
   FIELD Linenum AS INTEGER LABEL "Line Num" FORMAT ">>9" INITIAL "0"
```

Other XML-NODE-TYPE values let you specify that a value should be written out as an "ELEMENT", an XML "ATTRIBUTE", or as "TEXT". XML-NODE-TYPE is also a CHARACTER attribute that you can set at runtime on a buffer field.

Running writeOrderNested.p generates the following variant of the XML document into the file dsOrderNested.xml. You can see that the OrderLines are all nested within the Order, and that the OrderNum element is omitted from each OrderLine:

```
<?xml version="1.0" ?>
-<dsOrder xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
-<eOrder>
  <Ordernum>1</Ordernum>
  ...
  <CustName>Offside Hockey</CustName>
  <RepName>Robert Roller</RepName>
  <OrderTotal>4402.02</OrderTotal>
-<eOrderLine>
  <Linenum>1</Linenum>
  <Itemnum>54</Itemnum>
  ...
  </eOrderLine>
-<eOrderLine>
  <Linenum>2</Linenum>
  <Itemnum>55</Itemnum>
  ...
  </eOrderLine>
  ...
 </eOrderLine>
</eOrder>
</dsOrder>
```

At this time there is no corresponding way to get the READ-XML method to fill in foreign key fields (child fields in Data-Relations) with key field values from the parent. This will be addressed in a later product release. [Note: This is being treated as a bug and may possibly be corrected before this paper is published, allowing me to describe the amended behavior, which will probably be to have READ-XML automatically populate Data-

Relation child fields from NESTED XML if the XML for the child does not provide values for the foreign key element(s).]

You can code procedures that use WRITE-XML and READ-XML in place of procedures such as the saxread.p example shown in the white paper on *Using an Unmanaged Data Source*, in order to manage XML data from your Data Access Objects in an application designed in accordance with the OpenEdge Reference Architecture. The new language features make it much easier to read XML data into and write XML as output from your application.

Saving Context Data as XML

The white paper on *Context Management* describes how to create a ContextStore table in the database using a RAW field to hold each row in a temp-table. In addition, you can use the OpenEdge MEMPTR support to transfer the contents of an entire temp-table or ProDataSet to a single BLOB field.

This section describes an alternative to storing data in this raw format, by storing it as XML instead. The code is in the procedure writeOrderCLOB.p. Storing data as XML between OpenEdge sessions or between interactions involving multiple sessions avoids some of the complications of using the RAW format, which is very low-level and very sensitive to details of how buffers and their built-in attributes are defined internally. The *Context Management* white paper describes in detail these complications and the coding needed to account for them. There is a key difference between the context storing described in the *Context Management* paper and the technique of storing an entire temp-table or ProDataSet as an XML document. Rather than having a row in what this example calls the XMLContextStore table for each row in each temp-table, there is only one context row for the entire document, representing an entire set of related data. If the application needs to access the data for individual rows within the XML representation, a context manager can use the OpenEdge SAX parser support to retrieve and perhaps update specific subsets of an XML document.

Given that each row in this XMLContextStore table represents an entire set of data and not an individual row, the fields in the table are somewhat different from the example in the *Context Management* paper. These fields are used by the present example:

- **SessionID** – A first-level unique CHARACTER identifier that can be used to identify the client session or as any other meaningful key.
- **ContextID** – A second-level unique CHARACTER identifier that could be used to identify a particular interaction within a session.
- **ContextName** – A meaningful human-readable CHARACTER identifier for the context information
- **TimeStamp** – A DATETIME-TZ field that stores the date and time when the context was stored or last modified
- **DeleteTime** – A DATETIME-TZ field that stores the date and time when the context should be deleted for cleanup purposes
- **XMLData** – a CHARACTER large object (CLOB) field holding the data as XML

The code sample directory for this paper has a data definition file named xmlconte.df that you can use to add the XMLContextStore table to the Sports2000 database.

To transfer data to a CLOB field, you must first designate a LONGCHAR variable as the immediate target. LONGCHAR is the variable datatype that corresponds to a CLOB field in a table.

```
DEFINE VARIABLE lcXMLData  AS LONGCHAR   NO-UNDO.
```

The example also shows how to generate a unique CHARACTER ID for use as a context key in the table.

```
DEFINE VARIABLE cSessionID AS CHARACTER  NO-UNDO.
```

OpenEdge Release 10.1 provides a new built-in function, GENERATE-UUID, to generate a Universally Unique IDentifier as a 16-byte RAW value. You can turn this into a displayable 24-byte CHARACTER value by applying the BASE64-ENCODE function to it, then using the SUBSTRING function to remove the final two padding bytes from the resulting string, as shown in this line from the example procedure:

```
cSessionID = SUBSTRING(BASE64-ENCODE(GENERATE-UUID),1,22).
```

The result is a 22-character identifier (also called a GUID, or Globally Unique Identifier) that is guaranteed to be absolutely unique across all possible time and usage.
You can use GUIDs like this as keys for many purposes, including as primary keys of database tables, in place of values derived from database sequences. As long as the character value itself does not need to be easily readable, a GUID provide these advantages over a sequence value:

– It can be generated on any platform, without the need for a database connection.

– It is guaranteed to be absolutely unique, so there is no need to be concerned about key value conflicts in the event that related tables are combined in the future.

– The number of possible values is essentially limitless, so there is no need to be concerned about the possibility of an overflow.

– Given that the GUID is not intended to be a meaningful identifier in an everyday sense, the way a Customer number or an Order number can be, there is no need to anticipate changing a GUID and having to change all the other values in other tables that might be related to it.

– Because all GUIDs in all tables that use them are unique, you can take advantage of the fact that a GUID used as a primary key identifies a particular row even across all the tables in all the databases in the world. Thus, given a table identifier, a GUID provides you with a uniform way to access any row in that table. In this way you can create tables for information such as auditing data in which the combination of table name and GUID identifies a particular row in any database table, with no risk of duplication even when databases are combined.

The WRITE-XML method in writeOrderCLOB.p describes the target type as “LONGCHAR” instead of “FILE”, and names the LONGCHAR variable that is the target for the write. The third argument is FALSE to tell OpenEdge not to bother with white space formatting, since the XML is to be written out as a single string.

```
DATASET dsorder:WRITE-XML
    ("LONGCHAR",
     lcXMLData,
     FALSE).   /* unformatted */
```

To show how you can save the data as persistent context, to be shared between sessions, the procedure creates a row in the XMLContextStore database table, sets the Session ID key field, and copies the LONGCHAR value to the XMLData CLOB field, along with a time stamp and default delete time. The second-level ContextID key is not used in this example.

```
    CREATE XMLContextStore.
    ASSIGN XMLContextStore.SessionID = cSessionID
           XMLContextStore.ContextName = "Order 1"
           XMLContextStore.XMLData = lcXMLData
```

```
            XMLContextStore.TimeStamp = NOW
            XMLContextStore.DeleteTime =     /* Same time tomorrow */
                  ADD-INTERVAL(XMLContextStore.TimeStamp, 1, ”days”).
```

To demonstrate that you can retrieve the data in the same way, the procedure locates the right context table row using the Session ID, and copies it back into the LONGCHAR variable.

```
    FIND XMLContextStore WHERE XMLContextStore.SessionID = cSessionID.
    lcXMLData = XMLContextStore.XMLData.
```

Finally, the READ-XML method uses the LONGCHAR source type to load the data back into the ProDataSet:

```
DATASET dsOrder:READ-XML
    ("LONGCHAR",
     lcXMLData,
     "EMPTY",   /* read mode */
     ?,         /* schema location */
     FALSE).    /* override-default-mapping */
```

Summary

This paper has introduced to you several important language features new to OpenEdge 10.1 to assist in converting data between OpenEdge native format in temp-tables or ProDataSets, and standard XML format. You can use XML as a medium of data interchange between OpenEdge and other applications. In an application that is structured according to the OpenEdge Reference Architecture, you can enclose code to read and write XML in Data Access Objects that treat the XML as an unmanaged data store. You can also use XML as a way to persist application data or application context information to help you in connecting the pieces of a distributed application.

Corporate Headquarters
Progress Software Corporation, 14 Oak Park Drive, Bedford, MA 01730 USA
Tel: 781 280 4000 Fax: 781 280 4095

Europe/Middle East/Africa Headquarters
Progress Software Europe B.V. Schorpioenstraat 67 3067 GG Rotterdam, The Netherlands
Tel: 31 10 286 5700 Fax: 31 10 286 5777

Latin American Headquarters
Progress Software Corporation, 2255 Glades Road, One Boca Place, Suite 300 E, Boca Raton, FL 33431 USA
Tel: 561 998 2244 Fax: 561 998 1573

Asia/Pacific Headquarters
Progress Software Pty. Ltd., 1911 Malvern Road, Malvern East, 3145, Australia
Tel: 61 39 885 0544 Fax: 61 39 885 9473

PROGRESS SOFTWARE

www.progress.com